
*Procesamiento de
Imágenes*

Introducción a la detección y clasificación de objetos

Isaac Fermin Llauger



¿Qué es detectar y qué es clasificar objetos?



- Detectar objetos es encontrar dónde hay algo en la imagen: localizar su posición y tamaño (por ejemplo, dibujar un recuadro alrededor de una cara o un carro).
- Clasificar objetos es decidir qué es eso que encontramos: si es una persona, un carro, un perro, un semáforo, etc.

Ejemplo

Desbloqueo facial del celular

Detección de autos y peatones

CONCEPTOS CLAVE

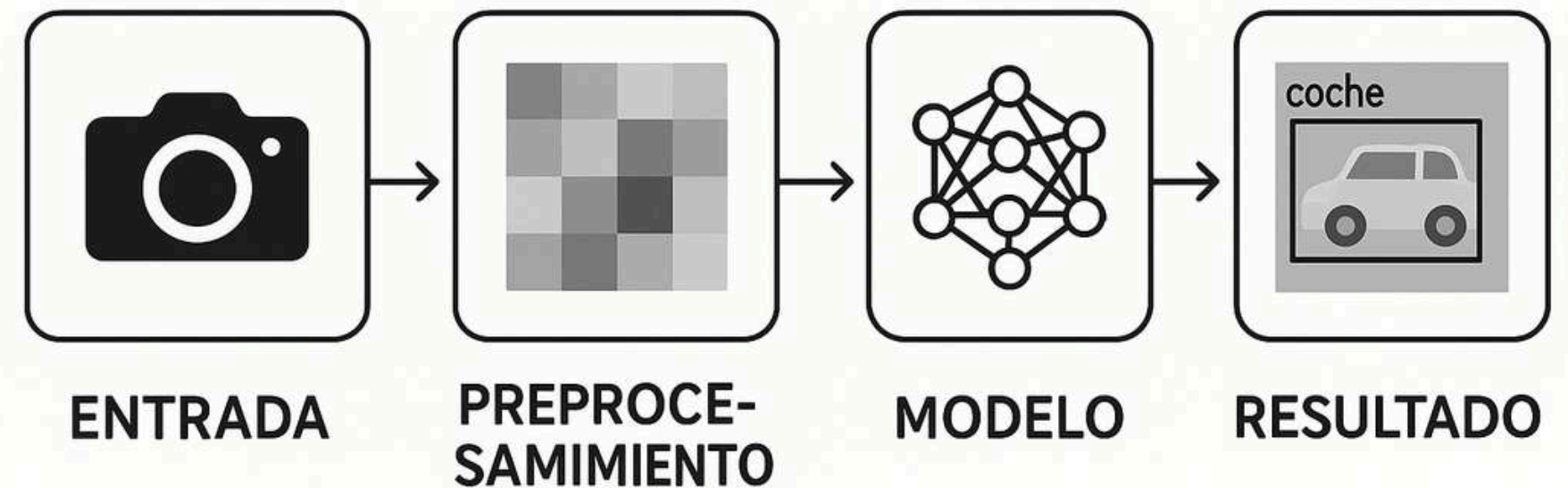
Clasificación vs detección de objetos

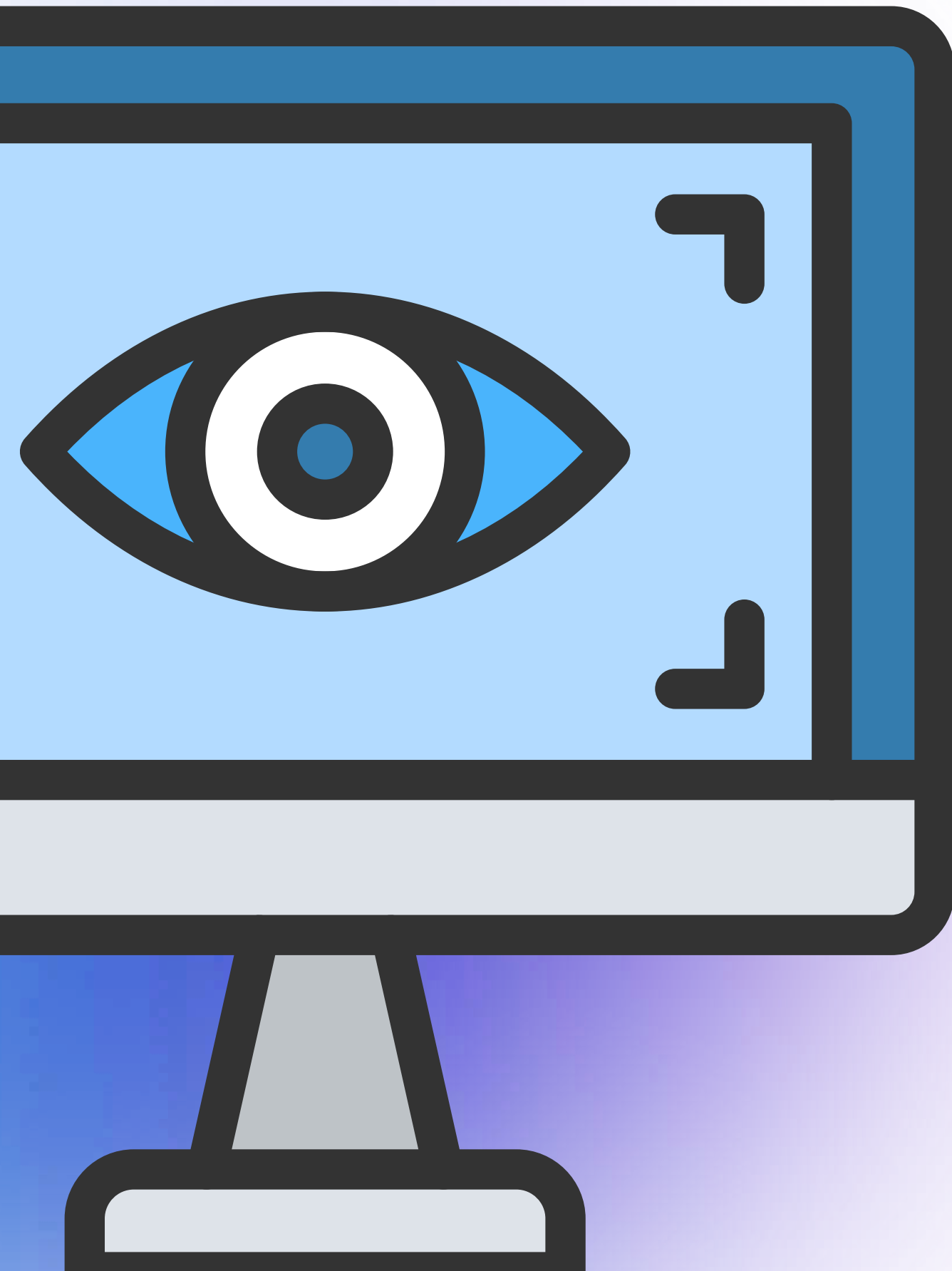
- Clasificación de imágenes: el modelo recibe una imagen completa y devuelve una sola etiqueta para toda la imagen (ej.: “perro”, “auto”, “semáforo”).
- Detección de objetos: el modelo indica qué hay y dónde está cada cosa, dibujando cajas (bounding boxes) alrededor de los objetos y asignando una etiqueta a cada uno.

Breve repaso de conceptos

- Una imagen digital está formada por muchos píxeles, que son los “puntitos” de color.
- En escala de grises, cada píxel solo tiene intensidad (de negro a blanco), lo que simplifica muchos cálculos.
- Un ROI (Region of Interest) es una zona específica de la imagen donde queremos enfocarnos, por ejemplo, la parte donde podría estar una cara o un carro.

Mini esquema del pipeline



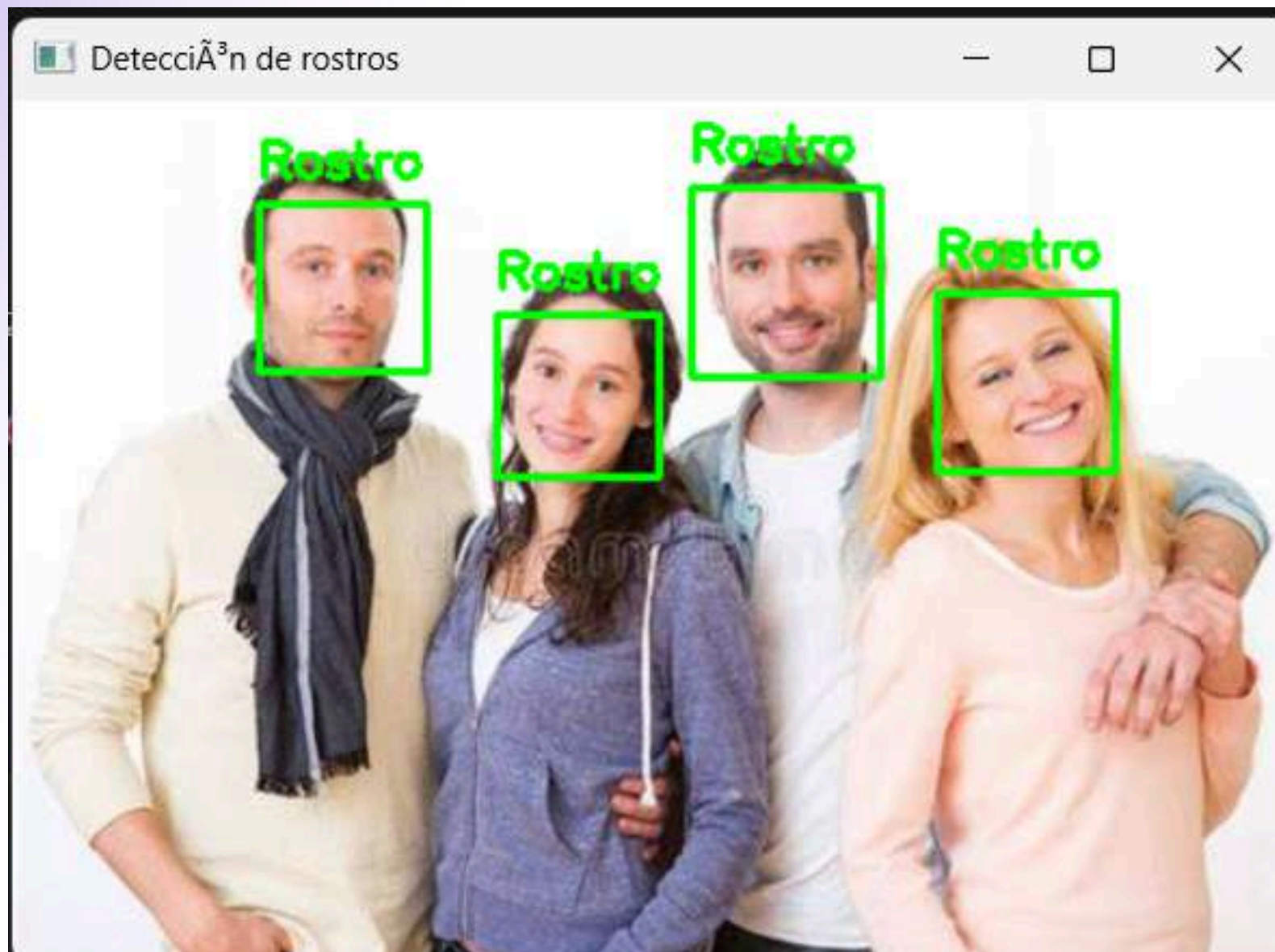


MÉTODOS CON OPENCV (VISIÓN GENERAL)

- Haar Cascades
- HOG + SVM
- CNN
- YOLO
- SSD

EJEMPLO 1: DETECCIÓN CON HAAR CASCADE

Objetivo: detectar rostros en una imagen



```
# 1. Construir la ruta del cascade
haar_dir = cv2.data.harcascades
cascade_file = "haarcascade_frontalface_default.xml"
cascade_path = os.path.join(haar_dir, cascade_file)

# 2. Cargar el cascade con .load()
face_cascade = cv2.CascadeClassifier()

● loaded = face_cascade.load(cascade_path)

# 3. Cargar la imagen
img = cv2.imread("grupo.jpeg")
print("shape:", img.shape)

# 4. Pasar a gris
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 5. Detectar rostros
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(40, 40)
)

print("Rostros detectados:", len(faces))

# 6. Dibujar rectángulos y texto

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.putText(img, "Rostro", (x, y-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

cv2.imshow("Detección de rostros", img)
```


EJEMPLO 2: CLASIFICACIÓN SENCILLA

Clasificar un perro usando un modelo de Deep Learning preentrenado.



```
1/1 ————— 1s 1s/step
Etiqueta predicha: golden_retriever (0.83)
```

```
import cv2
import numpy as np

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import (
    preprocess_input,
    decode_predictions
)
```

```
# 1. Cargar modelo preentrenado
model = MobileNetV2(weights="imagenet")

# 2. Cargar imagen con OpenCV
img = cv2.imread("dog.jpg")

# 3. Preprocesar: BGR -> RGB, resize, normalizar
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_resized = cv2.resize(img_rgb, (224, 224)) # tamaño esperado por MobileNetV2

x = img_resized.astype("float32")
x = np.expand_dims(x, axis=0)
x = preprocess_input(x) # normalización propia del modelo

# 4. Pasar por el modelo y obtener predicción
preds = model.predict(x)
decoded = decode_predictions(preds, top=1)[0][0]
class_id = decoded[0]
class_name = decoded[1]
prob = decoded[2]

label = f"{class_name} ({prob:.2f})"
print("Etiqueta predicha:", label)

# 5. Mostrar resultado con la etiqueta sobre la imagen
output = img.copy()
cv2.putText(output, label, (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

cv2.imshow("Clasificación de imagen", output)
```

DESAFÍOS Y LIMITACIONES

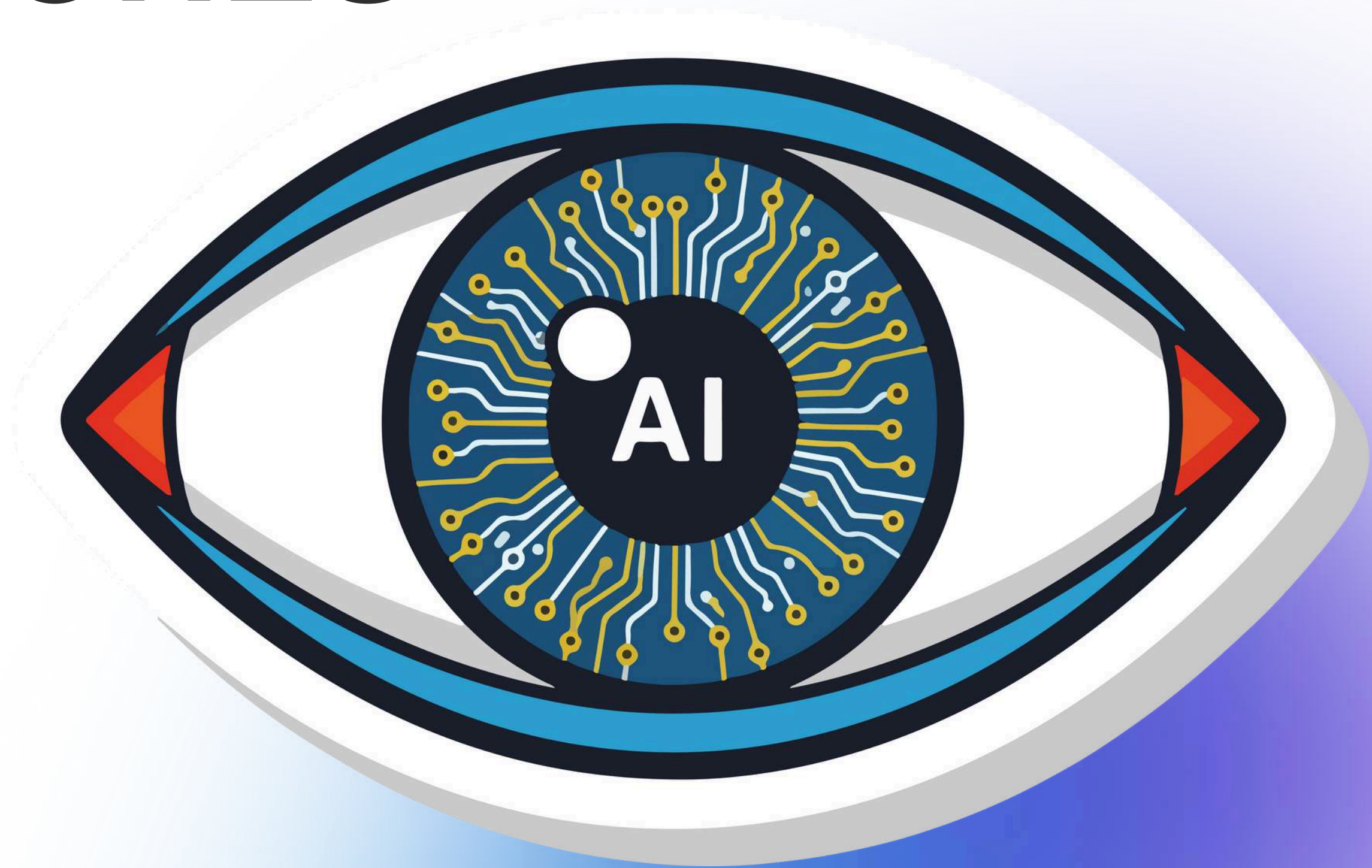


- ILUMINACIÓN
- SOMBRAS
- RUIDO
- OBJETOS PARCIALMENTE TAPADOS
- ESCALA
- ROTACIÓN Y PERSPECTIVA
- TIEMPO DE EJECUCIÓN
- IMPORTANCIA DE BUENOS DATOS DE ENTRENAMIENTO

CONCLUSIONES

Aplicaciones reales

- Seguridad: cámaras que detectan rostros, intrusos o comportamientos sospechosos.
- Transporte: detección de autos, peatones y señales de tráfico en sistemas de asistencia al conductor.
- Industria: conteo de productos en una banda, control de calidad, detección de defectos.
- Agricultura: detección de plagas, conteo de frutos, monitoreo de cultivos con drones.



!Gracias!